

ESPECIFICAÇÃO DE ARQUITETURA

Conector Multicanal Híbrido de Marketplaces

Credenciamento Global e Individual · Criptografia em Repouso via Fernet ·
Roteamento de Webhooks por Segmentação na Origem

DOCUMENTO	Especificação Técnica de Arquitetura
CÓDIGO	ARQ-MKT-2026-01
MÓDULO	apps/marketplaces
DESTINATÁRIO	Equipe de Engenharia
VERSÃO	1.0
DATA	13 de setembro de 2026

SUMÁRIO

1. Sumário Executivo
2. Escopo e Premissas
3. Decisão de Localização na Interface
4. Visão Arquitetural Geral
5. Modelo de Dados Conceitual
6. Padrão de Criptografia em Repouso (Fernet)
7. Ciclo de Vida: Callback OAuth2
8. Ciclo de Vida: Webhooks — Modelos Comparativos
9. Decisão Arquitetural: Segmentação na Origem
10. Token Público de Roteamento (Webhook UUID)
11. Considerações Multicanal
12. Requisitos Funcionais de Interface (UI/UX)
13. Resumo Consolidado de Decisões
14. Itens em Aberto e Próximos Passos

1. Sumário Executivo

Esta especificação documenta a arquitetura para suportar, dentro do módulo de **Marketplaces** do Hub, dois modos de credenciamento de aplicação — **Global** (SaaS compartilhado) e **Individual** (aplicação própria do lojista) — para múltiplos canais de venda (Mercado Livre, Shopee, Amazon, Magalu, Shein), sem alterar a estrutura da Navbar principal do sistema.

O documento consolida quatro decisões centrais:

- **Localização:** o recurso vive dentro do formulário de configuração de conta em Marketplaces, e não como novo item de Navbar.
- **Segurança:** todos os segredos de aplicação e tokens de sessão são cifrados em repouso via Fernet.
- **Callback OAuth2:** permanece como endpoint único e centralizado por canal, com identificação do tenant via parâmetro `state` assinado.
- **Webhooks:** adota roteamento híbrido — endpoint único por canal quando a aplicação é Global (identificação via `payload`), e endpoint com token UUID exclusivo por conta quando a aplicação é Individual (identificação e validação na origem, antes do parsing do corpo).

2. Escopo e Premissas

- Módulo afetado: **apps/marketplaces** — backend de conectores e formulário de configuração de conta.
- A Navbar do Hub (Início, Lojas, Categorias, Produtos, Marketplaces, Anúncios, Pedidos, Simulador, Logs, Usuários) permanece **inalterada**.
- A arquitetura é **agnóstica de canal**: aplica-se igualmente a Mercado Livre, Shopee, Amazon, Magalu, Shein e futuros canais a integrar.
- Cada conta de marketplace (MarketplaceConta) pertence a uma Loja/Tenant e opera de forma independente das demais contas do mesmo canal.
- Este documento trata de **arquitetura e requisitos funcionais**; a implementação de código é de responsabilidade do time de engenharia.

3. Decisão de Localização na Interface

3.1 Fora da Navbar

A Navbar representa a esteira macro e cronológica do negócio (Loja → Catálogo → Canal → Venda). Configuração de credenciais de API é um detalhe técnico do conector e não uma etapa da jornada operacional — portanto não recebe item próprio de navegação.

3.2 Dentro de Marketplaces

A configuração reside no formulário de **Cadastro/Edição de Conta**, dentro do menu Marketplaces já existente, posicionada junto ao botão de autenticação OAuth2.0 — pois o fluxo de OAuth depende diretamente de qual `client_id/client_secret` será usado no momento da montagem da URL de autorização.

Tipo de Conector	Onde se Configura	Comportamento do Botão OAuth
Aplicação Global (SaaS)	Variáveis de ambiente do Hub (.env)	Usuário apenas clica em “Autorizar”
Aplicação Individual (Tenant)	Formulário de Edição de Conta, em Marketplaces	Usuário informa App ID e Secret, salva, e então clica no mesmo botão OAuth

4. Visão Arquitetural Geral

O diagrama abaixo resume o fluxo híbrido: a origem das credenciais depende do modo selecionado na conta, mas ambos os caminhos convergem para o mesmo módulo de criptografia antes de qualquer persistência em banco.

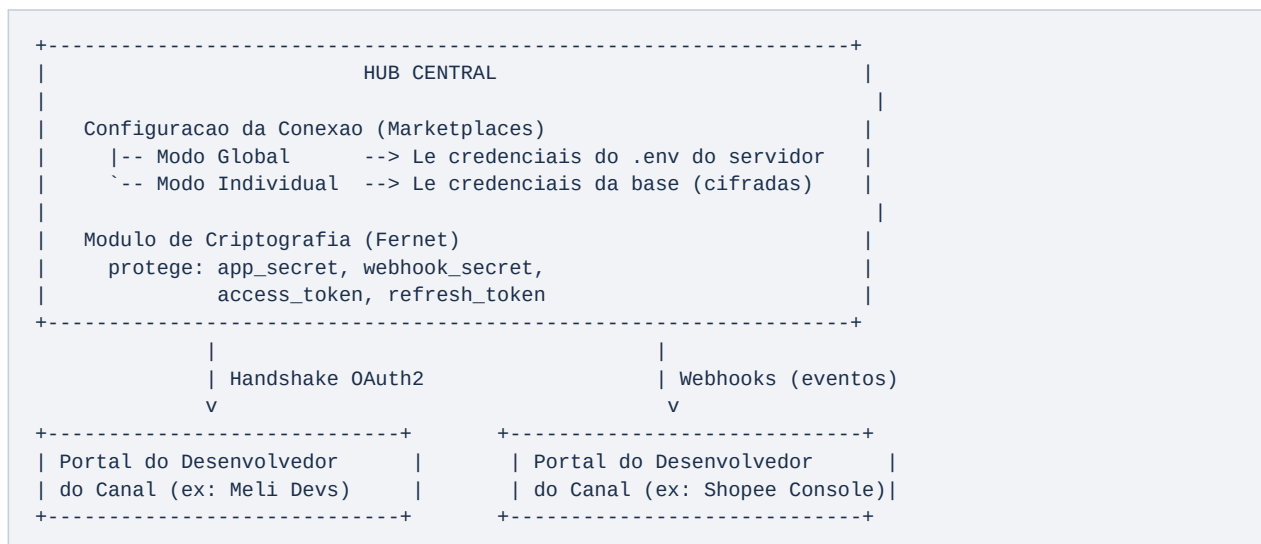


Figura 1 — Fluxo geral de credenciamento e proteção de segredos.

5. Modelo de Dados Conceitual

A entidade central do módulo, MarketplaceConta, representa a conexão de uma Loja/Tenant com um canal de venda específico. Ela encapsula tanto o modo de credenciamento quanto os segredos protegidos por criptografia.

Atributo	Tipo Conceitual	Restrição / Tratamento	Finalidade
loja	Referência (FK) para Loja/Tenant	Obrigatório; isolamento por cascata	Amarração de isolamento multitenant
canal	Enumeração	MELI, SHOPEE, AMAZON, MAGALU, SHEIN	Motor de despacho de regras da API externa
nome_identificador	Texto curto	Obrigatório	Rótulo amigável da conta (ex.: "Matriz SP")
tipo_aplicacao	Enumeração	GLOBAL ou INDIVIDUAL	Alternador de resolução de credenciais
app_key_or_id	Texto	Obrigatório se Individual	Identificador público (Client ID / Partner ID)
app_secret	Campo cifrado (Fernet)	Obrigatório se Individual	Segredo da aplicação própria
webhook_secret	Campo cifrado (Fernet)	Recomendado se Individual	Validação de assinatura HMAC
webhook_token	Identificador público (UUID)	Gerado automaticamente; indexado	Roteamento da URL de webhook (só Individual)
access_token	Campo cifrado (Fernet)	Preenchido pós-OAuth	Chave de sessão para chamadas à API
refresh_token	Campo cifrado (Fernet)	Preenchido pós-OAuth	Renovação de autorização
token_expira_em	Data/Hora	Opcional	Controle proativo de expiração
seller_id_remoto	Texto	Indexado	ID do vendedor no canal (Seller/Shop/Merchant ID)
ativo	Booleano	Padrão: verdadeiro	Estado de ativação da sincronização

6. Padrão de Criptografia em Repouso (Fernet)

Todo segredo persistido na camada relacional deve ser indecifrável sem a posse da chave mestra do sistema. Adota-se o esquema **Fernet** — criptografia simétrica autenticada (AES-128 em modo CBC, com autenticação HMAC e chave derivada de 256 bits).

Gestão de chave

- A chave mestra é obtida de uma variável de ambiente dedicada (FIELD_ENCRYPTION_KEY).
- Na ausência dessa variável, o sistema deriva uma chave compatível a partir do SECRET_KEY da aplicação via função hash SHA-256, garantindo que sempre exista uma chave válida — embora o uso de chave dedicada seja fortemente recomendado em produção.

Escopo de proteção obrigatório

- Segredo da aplicação (app_secret)
- Chave de assinatura do webhook (webhook_secret)
- Token de acesso (access_token)
- Token de renovação (refresh_token)

Benefício de segurança: em caso de vazamento ou exfiltração da base PostgreSQL, nenhum desses valores é exposto em texto plano — um invasor precisaria adicionalmente da chave mestra do ambiente de aplicação, que fica fora do banco de dados.

7. Ciclo de Vida: Callback OAuth2

O endpoint de Callback (Redirect URI) do fluxo OAuth2 (RFC 6749) é, por exigência dos provedores de canal, **obrigatoriamente único e centralizado** — não comporta segmentação por Seller/Tenant na própria URL.

Por quê

- Os consoles de desenvolvedor dos canais (Meli Devs, Shopee Open Platform, Amazon Seller Central) exigem pré-cadastro de uma lista estrita de URLs de redirecionamento autorizadas.
- Coringas (wildcards), como `https://hub.com/oauth/*/callback`, não são aceitos por nenhum dos provedores avaliados.

Estrutura canônica

```
https://<dominio>/marketplaces/<canal>/callback/
```

Identificação do tenant: como a URL é fixa, o Hub identifica a loja/conta de destino por meio do parâmetro state — um token assinado (HMAC) gerado no momento em que a autorização é disparada, contendo o identificador da loja e da conta. O canal devolve esse mesmo state junto com o code de autorização; o Hub o decodifica para localizar o registro exato onde os tokens (access_token, refresh_token) serão gravados, já cifrados via Fernet.

8. Ciclo de Vida: Webhooks — Modelos Comparativos

Diferente do Callback, o endpoint de Webhook (recepção de eventos assíncronos) admite dois modelos arquiteturais, cada um com trade-offs distintos.

Critério	Modelo A — Endpoint único por canal	Modelo B — Endpoint distinto por conta
Formato da URL	/api/v1/webhooks/	/api/v1/webhooks///
Compat. com App Global	Nativa — uma única URL central para todos os lojistas	Incompatível — provedores só permitem uma URL por App ID
Compat. com App Individual	Compatível	Compatível — o lojista cadastra a própria URL
Identificação do Tenant	Via payload (user_id / seller_id / shop_id)	Via URL (identificador extraído antes do parse do corpo)
Validação de assinatura	Corpo lido antes de localizar o webhook_secret	Segredo localizado pela URL antes de ler o corpo
Complexidade de roteamento	Mínima — endpoint fixo por canal	Maior — gestão de identificadores dinâmicos por conta

9. Decisão Arquitetural: Segmentação na Origem

A diretriz adotada combina os dois modelos de forma hierárquica, segundo o tipo de aplicação da conta — uma abordagem híbrida que preserva a compatibilidade com aplicações Globais e reforça a segurança das aplicações Individuais.

9.1 Modo Global → Endpoint único por canal

- Motivo: o provedor permite cadastrar apenas uma URL de notificação por App ID central do Hub.
- Identificação do tenant: extraída do payload (user_id/seller_id), com busca indexada na base.
- Segredo utilizado na validação: o segredo Global, definido em variável de ambiente.

9.2 Modo Individual → Endpoint com token UUID exclusivo

- Cada conta recebe um identificador público (webhook_token) distinto da chave primária sequencial interna, evitando exposição de IDs incrementais do banco.
- A URL exposta ao lojista já contém esse token: /api/v1/webhooks/<canal>/<uuid>/
- O UUID permite localizar a conta e o webhook_secret correspondente antes de processar qualquer conteúdo do corpo da requisição.

9.3 Fluxo de Segurança Fail-Fast — Modo Individual

```
Requisicao POST recebida
|
v
/api/v1/webhooks/<canal>/<uuid>/
|
|-- 1. Busca a conta pelo UUID (indexado)
|    nao encontrou? --> HTTP 404 imediato
|
|-- 2. Descriptografa o webhook_secret (Fernet)
|
|-- 3. Valida assinatura HMAC / SHA-256
|    invalida? --> HTTP 401/403 (aborta antes do parse do corpo)
|
|-- 4. Payload autenticado --> enfileira evento para processamento
```

Figura 2 — Curto-circuito de segurança (fail-fast) para aplicação Individual.

9.4 Simetria de Fail-Fast — Modo Global

Mesmo sem um identificador na URL, o modo Global também deve validar a assinatura antes de processar o corpo — a diferença é que o segredo usado é o Global, já disponível em memória da aplicação, sem necessidade de consulta prévia ao banco. Somente após a assinatura ser validada é que o Hub lê o corpo para extrair user_id/seller_id e localizar a conta correspondente.

```
Requisicao POST recebida
|
v
/api/v1/webhooks/<canal>/
|
|-- 1. Valida assinatura HMAC com o segredo Global (.env)
|      invalida? --> HTTP 401/403 (aborta antes do parse do corpo)
|
|-- 2. Payload autenticado --> le user_id / seller_id do corpo
|
|-- 3. Busca a conta correspondente na base --> enfileira evento
```

Figura 3 — Curto-circuito de segurança (fail-fast) para aplicação Global.

Matriz de Resolução de Rotas

Modo da Aplicação	Formato da URL Exposta	Estratégia de Identificação	Momento da Validação
Global (SaaS)	/api/v1/webhooks//	Pelo user_id/seller_id no corpo do payload	Antes da leitura do corpo, com o segredo Global
Individual (Própria)	/api/v1/webhooks///	Direta pela URL (identificador de origem)	Antes da leitura do corpo, com o segredo da conta

10. Token Público de Roteamento (Webhook UUID)

Para suportar a segmentação na origem, cada conta de marketplace Individual recebe um identificador público e não sequencial (UUID), desacoplado da chave primária interna do banco de dados.

- É gerado automaticamente na criação da conta e indexado para busca rápida.
- Compõe a URL de webhook exibida ao lojista, acompanhada de botão de cópia rápida.
- Pode ser regenerado sob demanda — a interface deve oferecer um botão de “Regenerar Token do Webhook” para cenários em que o lojista suspeite de vazamento da URL pública, invalidando o token anterior e exigindo nova configuração no console do canal.

11. Considerações Multicanal

A arquitetura desacopla o motor de autorização e de eventos do canal específico, permitindo que Mercado Livre, Shopee, Amazon, Magalu e Shein coexistam sob o mesmo modelo relacional e o mesmo padrão de segurança. Um registro central de configuração por canal define rótulos de campos, caminhos de endpoint e requisitos de segredo.

Canal	Suporta Global	Rótulo do Identificador	Mecanismo de Notificação
Mercado Livre	Sim	App ID (Client ID)	Webhook HTTP (URL de notificação)
Shopee	Sim	Partner ID	Webhook HTTP (URL de notificação)
Amazon SP-API	Sim	LWA Client ID	Fila / Evento (SQS ou EventBridge)
Magalu	Sim	Client ID	Webhook HTTP (URL de notificação)

Ponto de atenção — Amazon SP-API: diferente dos demais canais, a Amazon não notifica eventos via POST direto a uma URL cadastrada em console — a integração ocorre por assinatura de fila (Amazon SQS) ou barramento de eventos (EventBridge). Por isso, o registro de configuração por canal deve conter um campo que descreva o **mecanismo de notificação** (ex.: HTTP_WEBHOOK vs. FILA_EVENTOS), permitindo que a interface adapte o bloco exibido: para canais HTTP, mostra URL com botão de cópia; para a Amazon, deve orientar o cadastro de uma fila/ARN em vez de uma URL, evitando um formulário que sugira uma ação incompatível com o provedor.

12. Requisitos Funcionais de Interface (UI/UX)

- **Localização estrutural:** o recurso pertence exclusivamente ao formulário de cadastro/edição de conta dentro de Marketplaces; nenhuma rota nova é criada na Navbar.
- **Comportamento condicional:** ao selecionar “Aplicação Global”, os campos de credenciais técnicas permanecem ocultos ou bloqueados; ao selecionar “Aplicação Própria / Individual”, o formulário expande a seção de credenciais e exibe as URLs de Callback e Webhook (esta já com o token UUID exclusivo), cada uma com botão de cópia rápida.
- **Mascaramento de segredos:** qualquer segredo já salvo (Client Secret, Webhook Secret, tokens) deve ser exibido mascarado por padrão, nunca em texto plano após o salvamento inicial.
- **Indicadores de status:** a tela exibe separadamente o Status de Autenticação e o Status do Webhook, permitindo diagnóstico rápido sem acessar logs.
- **Ação de regeneração:** botão dedicado para regenerar o token UUID do webhook em contas Individuais, para resposta a suspeita de vazamento.
- **Adaptação por canal:** rótulos de campos e o bloco de notificação (URL de webhook vs. orientação de fila/evento) se adaptam dinamicamente conforme o canal selecionado.

13. Resumo Consolidado de Decisões

Aspecto	Decisão
Localização na UI	Dentro de Marketplaces → Configuração de Conexão (não na Navbar)
Seletor de modo	Alternador Global / Individual por conta de marketplace
Modo Global — credenciais	Lidas do .env do Hub
Modo Global — webhook	Endpoint único por canal; identificação do tenant via payload
Modo Individual — credenciais	Informadas pelo lojista; cifradas com Fernet
Modo Individual — webhook	Endpoint com token UUID exclusivo por conta; identificação e validação na origem
Criptografia em repouso	Fernet para app_secret, webhook_secret, access_token, refresh_token
Callback OAuth2	Endpoint único por canal; tenant identificado via parâmetro state assinado
Multicanal	Registro central de configuração por canal (rótulos, caminhos, mecanismo de notificação)

14. Itens em Aberto e Próximos Passos

1. Definir o tratamento de interface específico para canais com mecanismo de notificação por fila/evento (Amazon SP-API), evitando reaproveitar o bloco padrão de “Copiar URL de Webhook”.
2. Avaliar um indicador visual (badge) no item “Marketplaces” da Navbar quando alguma conexão estiver com status de erro (autenticação expirada ou webhook inativo) — melhoria de UX não bloqueante para esta entrega.
3. Validar o processo de rotação da chave mestra (FIELD_ENCRYPTION_KEY) sem perda de acesso aos dados já cifrados.